

SD TCL - Alternate Key Indexes

Indexing a field so records can be found by its contents, and the step that is easy to skip.

An alternate key index lets SD find records by what is in a field instead of by record id. Five verbs make, populate, inspect and remove them.

SD folds case, so a command may be typed in either case. Commands are shown here in lower case. In the tables, *italics* mark something you supply and **bold** marks a word typed as it stands; braces mark an optional part.

Every listing on this page was produced by running it, on SD Core for Windows W1.0-0, against a six-record file whose field 1 held five `V` and one `K`.

Use `make.index`

That is the whole of this page's advice. `create.index` and `build.index` are two halves of what `make.index` does in one command, and **the half that is easy to miss is the second one**:

```
make.index zzak f2
```

```
Added index for F2
Building index 'F2'...
6 records processed
Populating index...
```

Use the two-step form only when you have a reason to separate them.

The five verbs

<code>create.index</code> <i>file field...</i>	define an index. Does not populate it
<code>build.index</code> <i>file field...</i> all	populate it from the records already there

<code>make.index</code> <i>file field...</i>	both, in one command
<code>list.index</code> <i>file field...</i>	what indexes the file has, and whether they work
<code>delete.index</code> <i>file field... all</i>	remove them

`create.index` and `make.index` also take:

<code>no.nulls</code>	do not index records whose field is empty
<code>pathname</code> <i>path</i>	put the index somewhere other than beside the file

Why the two-step form catches people

`create.index` records the definition and **walks nothing**:

```
create.index zzak f1
```

```
Added index for F1
```

That reports success, and the index it made matches no records at all. The proof is in `list.index`:

```
list.index zzak f1
```

```
Alternate key indices for file zzak
Number of indices = 1
Index name..... En Type Nulls S/M Fmt Field/Expression
F1                N  D    Yes  S   L   1
```

The `En` column is the one to read. `N` means the index is not enabled — it exists, it is listed, and it will not find anything. After `build.index` the same line reads `Y`:

```
F1                Y  D    Yes  S   L   1
```

So `list.index` is the check, and "the index is there" is not the same question as "the index works".

`build.index` needs the file to itself

```
build.index zzak f1
```

```
Cannot gain exclusive access to file
```

That is what you get if anything has the file open — including the command you just typed. `create.index` followed by `build.index` in the same session fails this way every time, because `create.index` leaves the file open. From a fresh session it succeeds:

```
Building index 'F1'...
6 records processed
Populating index...
```

`make.index` **does not have this problem**, which is the practical reason to prefer it.

A session that was killed rather than logged out also holds the file. Its entry stays in the user table, `listu` still lists it, and `build.index` is refused with the message above until it is cleared. If a build is refused and you are certain nothing is using the file, run `listu` before looking for anything more exotic.

Asking what a file has

```
list.index zzak f1 f2
```

```
Alternate key indices for file zzak
Number of indices = 2
Index name..... En Type Nulls S/M Fmt Field/Expression
F1                Y  D    Yes  S   L   1
F2                Y  D    Yes  S   L   2
```

COLUMN

`En` enabled — `Y` **only after a successful build**

`Type` `D` for a field, `I` for a computed expression

COLUMN

<code>Nulls</code>	whether empty values are indexed — <code>no.nulls</code> at creation is what changes it
<code>S/M</code>	single- or multi-valued
<code>Fmt</code>	the key's format code, as a dictionary entry carries it
<code>Field/Expression</code>	the field number, or the I-type expression

A file with none says so plainly:

```
File has no indices
```

Using an index

Nothing changes in how you write a query. An index makes a selection on that field faster; it does not add syntax and it does not change the answer:

```
count zzak with f1 = "V"
```

```
5 record(s) counted
```

That is the same command, and the same result, as before the index existed. **Index the fields you actually select on** — each index makes every write to the file a little more expensive, and one that is never used in a `with` clause costs and returns nothing.

Once built, an index maintains itself: ordinary writes and deletes keep it current with no further action. Reading an index from a program — `selectindex`, `indices()` — is in SD Basic - Alternate Key Indexes, which also has the detail on why **an already-open file variable never learns about a new index.**

Removing an index

```
delete.index zzak all
```

```
Deleted index F1
Deleted index F2
```

`all` removes every index on the file. Naming them individually also works — with one catch:

`delete.index` **matches the index name exactly, and index names are held in upper case.** `delete.index zzak f1` answers *"Unrecognised index name (f1)"* — on a file where `list.index zzak f1` had just found the index and printed it. `list.index` **folds case and** `delete.index` **does not**, so a name that lists perfectly well may still refuse to delete. Use `all`, or type the name exactly as `list.index` prints it.

Deleting the file removes its indexes with it.

Who has these verbs

All five are programmer verbs. A standard account has none of them, and does not need them — an index changes how a query runs, not how it is written, so a standard account gets the benefit of every index without being able to create or destroy one.

See also

SD Basic - Alternate Key Indexes · SD TCL - Files and Records · SD TCL - The Query Processor.
